```
fn main() -> i32 {
    println("Hello, Amiga!")
    return 0
}
```

NOVUS

Programmer's Reference Guide

A modern language for the Amiga

Version 0.1 | December 2025

For Amiga 68020+ | AmigaOS 2.0+

Novus Programmer's Reference Guide

Version 0.1 — December 2025

Novus is an open source project.
Visit `https://github.com/barryw/Novus` for source code and updates.

Typeset in Latin Modern using LaTeX.

Contents

Foreword **v**

1 Introduction **1**
1.1 What is Novus? . . . 1
1.1.1 Key Characteristics . . . 1
1.2 Why Novus Exists . . . 1
1.2.1 The Limits of C . . . 1
1.2.2 Modern Solutions, Amiga Constraints . . . 2
1.3 Design Philosophy . . . 2
1.3.1 Explicit Over Implicit . . . 2
1.3.2 Predictable Performance . . . 3
1.3.3 Respect the Machine . . . 3
1.3.4 Fail Fast, Fail Clearly . . . 3
1.4 Target Audience . . . 3
1.5 How to Read This Guide . . . 3

2 Getting Started **5**
2.1 Prerequisites . . . 5
2.2 Installation . . . 5
2.2.1 Building from Source . . . 5
2.2.2 Verifying Installation . . . 5
2.3 Your First Program . . . 6
2.3.1 Compiling . . . 6
2.3.2 Running on the Amiga . . . 6
2.4 Understanding the Toolchain . . . 6
2.4.1 Compiler Commands . . . 7
2.4.2 Compile Options . . . 7
2.4.3 Debug vs Release Builds . . . 7
2.5 Project Structure . . . 8
2.5.1 The project.toml File . . . 8
2.6 The Standard Library . . . 9
2.7 Running Tests . . . 9
2.7.1 Test Options . . . 10
2.8 Next Steps . . . 10

3 Language Basics **11**
3.1 Comments . . . 11
3.1.1 Single-Line Comments . . . 11
3.1.2 Multi-Line Comments . . . 11
3.2 Variables and Mutability . . . 11
3.2.1 Mutable Variables with `var` . . . 11
3.2.2 Immutable Bindings with `let` . . . 12
3.2.3 Compile-Time Constants with `const` . . . 12
3.2.4 Comparison . . . 12
3.2.5 Type Annotations . . . 12

3.3 Primitive Types . . . 12
3.3.1 Integer Types . . . 13
3.3.2 Floating-Point Types . . . 13
3.3.3 Fixed-Point Types . . . 13
3.3.4 Boolean Type . . . 13
3.4 Literals . . . 13
3.4.1 Integer Literals . . . 13
3.4.2 Floating-Point Literals . . . 14
3.4.3 String Literals . . . 14
3.4.4 F-Strings (Formatted Strings) . . . 15
3.4.5 Character Literals . . . 15
3.5 Operators . . . 15
3.5.1 Arithmetic Operators . . . 15
3.5.2 Comparison Operators . . . 16
3.5.3 Logical Operators . . . 16
3.5.4 Bitwise Operators . . . 16
3.5.5 Compound Assignment Operators . . . 17
3.5.6 Increment and Decrement Operators . . . 17
3.5.7 Range Operators . . . 18
3.6 Type Conversions . . . 18
3.6.1 C-Style Casts . . . 18
3.6.2 The `as` Keyword . . . 18
3.7 References and Pointers . . . 18
3.7.1 Immutable References . . . 18
3.7.2 Mutable References . . . 19
3.7.3 Raw Pointers . . . 19
3.7.4 Dereferencing . . . 19
3.8 Arrays . . . 19
3.8.1 Fixed-Size Arrays . . . 19
3.8.2 Array Literals . . . 19
3.8.3 Repeat Syntax . . . 20
3.8.4 Indexing . . . 20
3.9 Tuples . . . 20
3.9.1 Tuple Types . . . 20
3.9.2 Tuple Literals . . . 20
3.9.3 Destructuring . . . 20
3.9.4 Returning Multiple Values . . . 20
3.10 Expressions and Statements . . . 21
3.10.1 Expression vs Statement . . . 21
3.10.2 Block Expressions . . . 21
3.11 Summary . . . 21

4 Types **23**
4.1 Type System Overview . . . 23
4.2 Primitive Types . . . 23
4.3 Structs . . . 23
4.3.1 Declaring Structs . . . 24
4.3.2 Struct Visibility . . . 24
4.3.3 Creating Struct Instances . . . 24
4.3.4 Accessing Fields . . . 25
4.3.5 Struct Update Syntax . . . 25
4.3.6 Generic Structs . . . 25

4.3.7 Const Generic Parameters . . . 25
4.3.8 Where Clauses . . . 26
4.4 Implementation Blocks . . . 26
4.4.1 Self Parameter Variants . . . 26
4.4.2 Associated Functions . . . 27
4.4.3 Generic Implementations . . . 27
4.4.4 Multiple Impl Blocks . . . 28
4.5 Enums . . . 28
4.5.1 Unit Variants . . . 28
4.5.2 Tuple Variants . . . 29
4.5.3 Generic Enums . . . 29
4.5.4 Enum Limitations . . . 29
4.6 Traits . . . 30
4.6.1 Defining Traits . . . 30
4.6.2 Default Implementations . . . 30
4.6.3 Implementing Traits . . . 30
4.6.4 Built-in Traits . . . 31
4.6.5 Trait Bounds . . . 31
4.7 Pattern Matching . . . 32
4.7.1 Pattern Types . . . 32
4.7.2 Match Expressions . . . 33
4.7.3 If Let Expressions . . . 35
4.7.4 Let Else Expressions . . . 35
4.8 Option and Result . . . 36
4.8.1 Option<T> . . . 36
4.8.2 Result<T, E> . . . 37
4.8.3 The ? Operator . . . 38
4.9 Generics . . . 39
4.9.1 Generic Functions . . . 39
4.9.2 Generic Structs . . . 40
4.9.3 Const Generic Parameters . . . 40
4.9.4 Where Clauses . . . 40
4.9.5 Turbofish Syntax . . . 41
4.9.6 Generic Constraints in Practice . . . 41
4.10 Type Coercions and Casts . . . 42
4.10.1 Integer Widening . . . 42
4.10.2 Integer Narrowing . . . 42
4.10.3 Reference to Pointer . . . 42
4.10.4 Array to Pointer . . . 42
4.10.5 Explicit Casts . . . 43
4.11 Type Inference . . . 43
4.11.1 Local Variable Inference . . . 43
4.11.2 Function Return Type Inference . . . 43
4.11.3 Generic Type Inference . . . 43
4.12 Summary . . . 44

Foreword

The Amiga was ahead of its time. In 1985, while other personal computers offered beeps and static screens, the Amiga delivered pre-emptive multitasking, dedicated graphics and audio hardware, and a sophisticated operating system that wouldn't be matched by mainstream competitors for nearly a decade.

For many of us, the Amiga wasn't just a computer—it was where we learned to code, to create, to push hardware to its limits. The demoscene flourished. Games achieved visual and audio fidelity that seemed impossible. A generation of programmers cut their teeth on 68000 assembly, crafting tight loops that squeezed every cycle from the machine.

Then the platform faded from the mainstream. But it never truly died.

Today, a vibrant community keeps the Amiga alive. Original hardware is lovingly maintained and expanded. FPGA recreations achieve cycle-perfect accuracy. Emulation lets new generations experience what made these machines special. And remarkably, people still write software for them—not out of obligation, but out of genuine passion.

Yet the tools available to modern Amiga developers are largely the same ones we used thirty years ago. C compilers like VBCC and GCC produce excellent code, but the languages themselves haven't evolved. The patterns and safety mechanisms that modern systems programming takes for granted—sum types, pattern matching, resource management without garbage collection—remain out of reach.

Novus aims to change that.

This is not an attempt to modernize the Amiga by hiding what makes it special. Quite the opposite. Novus is designed to *embrace* the Amiga's architecture: its custom chips, its message-passing OS, its elegant hardware. The goal is to give developers modern ergonomics and safety while preserving—even enhancing—the direct hardware access that makes Amiga programming rewarding.

Whether you're a veteran who remembers waiting for Workbench to load from floppy, or a newcomer curious about this legendary platform, we hope Novus helps you write code that's safer, clearer, and more enjoyable—without sacrificing an ounce of the control that Amiga development demands.

New code for classic machines. Welcome to Novus.

December 2025

Chapter 1

Introduction

1.1 What is Novus?

Novus is a systems programming language designed specifically for the Amiga 68k platform. It transpiles to C99, which VBCC then compiles to native 68020+ machine code. The result is executables that integrate deeply with AmigaOS and run on real hardware and accurate emulators alike.

The name comes from the Latin word for "new"—reflecting the project's mission to bring modern language design to classic machines. Novus is not a port of an existing language, nor is it a lowest-common-denominator cross-platform tool. Every design decision is made with the Amiga in mind.

1.1.1 Key Characteristics

Native Performance Novus transpiles to C99, leveraging VBCC's mature 68k code generator to produce Amiga executables. There is no interpreter, no virtual machine, no runtime overhead beyond what you explicitly request.

Modern Safety The type system catches errors at compile time. `Result` and `Option` types make error handling explicit. Resource cleanup is deterministic via `defer` blocks. You get safety without garbage collection.

Direct Hardware Access Novus doesn't hide the Amiga's hardware behind abstractions you can't escape. Custom chip registers, Exec library calls, copper lists, blitter operations—all are first-class citizens with typed, safe interfaces.

Readable Output The generated C code is clean and readable. When you need to understand what the compiler is doing—and on the Amiga, you often do—the output won't fight you. You can also emit assembly via `-emit-asm` to see exactly what VBCC produces.

Amiga-Native Async The `async`/`await` model maps directly to AmigaOS primitives: Exec signals and message ports. No hidden threads, no runtime scheduler—just the OS facilities the Amiga provides.

1.2 Why Novus Exists

The Amiga platform has capable C compilers. VBCC, in particular, generates excellent code and remains actively maintained. So why create a new language?

1.2.1 The Limits of C

C was revolutionary in 1972. It remains useful today precisely because it's minimal—a portable assembler that gets out of your way. But that minimalism has costs:

- **No sum types.** Representing "this value is either an error or a result" requires conventions that the compiler can't enforce.
- **No built-in error handling.** Return codes are easily ignored. Exceptions don't exist. Every function call is an opportunity for silent failure.
- **Manual resource management.** Matching every `AllocMem` with a `FreeMem`, every `OpenLibrary` with a `CloseLibrary`, every `Lock` with an `UnLock`—across all control flow paths, including error cases.
- **Primitive module system.** Header files and `#include` are textual substitution. Name collisions, include order dependencies, and compile time explosion are facts of life.
- **No metaprogramming.** The preprocessor is powerful but crude. Anything beyond simple macros quickly becomes write-only code.

These aren't theoretical concerns. They're the source of real bugs in real Amiga software—memory leaks, use-after-free, null pointer crashes, resource exhaustion from unclosed libraries.

1.2.2 Modern Solutions, Amiga Constraints

Languages like Rust address these problems elegantly. But Rust targets modern systems with megabytes of RAM, gigahertz CPUs, and operating systems that provide virtual memory and threads. Its runtime assumptions don't map to the Amiga.

Novus takes a different approach: provide modern ergonomics within Amiga constraints.

- `Result[T, E]` and `Option[T]` are zero-cost at runtime—they're just tagged unions that the compiler understands.
- `defer` blocks compile to straightforward cleanup code inserted at scope exit. No hidden allocations, no unwinding tables.
- Pattern matching compiles to efficient jump tables or comparison chains as appropriate.
- The module system is compile-time only. No runtime symbol resolution, no dynamic linking overhead.
- Fixed-point math uses inline 68020+ instructions, not floating-point emulation.

The result is a language that *feels* modern but *runs* like hand-tuned assembly.

1.3 Design Philosophy

Novus follows several core principles that inform every language decision:

1.3.1 Explicit Over Implicit

If something allocates memory, the code should say so. If something can fail, the type should reflect it. If a function has side effects, they should be visible at the call site.

This doesn't mean verbose—Novus has type inference, sensible defaults, and concise syntax. But it means no hidden costs, no action at a distance, no "magic" that obscures what the machine is actually doing.

1.3.2 Predictable Performance

On a 7 MHz 68000, every cycle matters. On a 50 MHz 68060, you have more headroom—but you still can't afford hidden allocations in a tight loop or cache-hostile memory access patterns.

Novus gives you control. The compiler won't secretly box values, won't insert invisible runtime checks in release builds, won't generate code that's impossible to reason about.

1.3.3 Respect the Machine

The Amiga isn't a Unix workstation. It has custom chips that do things no other hardware can do. Its operating system uses message passing and signals, not files and processes. Its memory model is flat and physical.

Novus doesn't pretend otherwise. The standard library wraps AmigaOS idiomatically—`Result` types for operations that can fail, handles for resources that need cleanup—but it doesn't try to be POSIX. Amiga code should look like Amiga code.

1.3.4 Fail Fast, Fail Clearly

When something goes wrong, you should know immediately and know exactly what happened. Compile-time errors are better than runtime errors. Runtime errors with clear messages are better than silent corruption.

In debug builds, Novus checks array bounds, validates pointers, and panics with meaningful diagnostics. In release builds, those checks can be removed—but the type system still prevents the errors that checks would catch.

1.4 Target Audience

This guide assumes you're a working programmer. You should be comfortable with:

- Systems programming concepts: pointers, memory allocation, stack vs. heap
- Basic 68k architecture: registers, addressing modes, the supervisor/user split
- AmigaOS fundamentals: libraries, Exec, DOS, the message-passing model

Prior experience with Rust, Swift, or other modern systems languages will make Novus feel familiar, but it's not required. If you've written C for the Amiga, you have all the background you need.

1.5 How to Read This Guide

The guide is organized in layers:

Part I: Language Fundamentals covers the core language—syntax, types, control flow, functions, and modules. Read this first.

Part II: Standard Library documents the `std` packages: collections, strings, memory management, and Amiga OS bindings.

Part III: Advanced Topics covers async programming, inline assembly, FFI with existing C code, and performance optimization.

Appendices provide quick references, grammar summaries, and migration guides from C.

Code examples are designed to be complete and runnable. When a snippet requires context, we provide it. When we omit boilerplate, we say so.

Let's begin.

Chapter 2

Getting Started

> *"The journey of a thousand miles begins with a single step."*
> —Lao Tzu

This chapter walks you through installing the Novus compiler, writing your first program, and understanding the compilation pipeline. By the end, you'll have a working executable running on your Amiga (or emulator).

2.1 Prerequisites

Before installing Novus, ensure you have:

- **.NET 9.0 or later** — The Novus compiler is written in C# and requires the .NET runtime. Download from `https://dotnet.microsoft.com`
- **VBCC toolchain** — Novus uses VBCC's assembler (`vasmm68k_mot`) and linker (`vlink`) to produce Amiga executables. The toolchain is vendored in the repository under `vendor/vbcc`, or you can specify a custom installation via the `-vbcc-path` option.
- **An Amiga or emulator** — To run your compiled programs. FS-UAE, WinUAE, or real hardware all work. A 68020+ configuration with AmigaOS 2.0 or later is required.

2.2 Installation

2.2.1 Building from Source

Clone the repository and build:

```
git clone https://github.com/your-repo/novus.git
cd novus
dotnet build
```

The compiler will be available at `Novus/bin/Debug/net9.0/novus`.
For a release build with optimizations:

```
dotnet build -c Release
```

2.2.2 Verifying Installation

Check that the compiler runs:

```
./Novus/bin/Debug/net9.0/novus --help
```

You should see the available commands and options.

2.3 Your First Program

Create a file named `hello.novus`:

```
from std::io::file import println

fn main() -> i32 {
    println("Hello, Amiga!")
    return 0
}
```

Let's break this down:

- `from std::io::file import println` — Imports the `println` function from the standard library. Unlike some languages, Novus requires explicit imports.
- `fn main() -> i32` — Declares the entry point. All Novus programs must have a `main` function that returns an `i32` (the exit code).
- `println("Hello, Amiga!")` — Prints the message to standard output.
- `return 0` — Returns success (zero) to the operating system.

2.3.1 Compiling

Compile to an Amiga executable:

```
novus compile hello.novus -o hello
```

This produces an executable named `hello` in Amiga HUNK format. Without the `-o` flag, the output would default to `a.out`.

2.3.2 Running on the Amiga

Transfer the executable to your Amiga (via shared folder, network, or disk image) and run it from the Shell:

```
1.Workbench:> hello
Hello, Amiga!
```

Congratulations—you've run your first Novus program!

2.4 Understanding the Toolchain

The Novus compilation pipeline has several stages:

1. **Parsing** — Source code is parsed into an Abstract Syntax Tree (AST)
2. **Semantic Analysis** — Types are checked, names are resolved, and the Intermediate Representation (IR) is built
3. **C Code Generation** — The IR is lowered to C99 code targeting VBCC
4. **Compilation** — VBCC's C compiler (`vc`) compiles the C code directly to 68k object files

5. **Assembly** — Additional assembly files (runtime, library stubs) are assembled with `vasm`
6. **Linking** — `vlink` combines all object files with the runtime and produces the final executable

This pipeline means you get the benefits of VBCC's mature 68k code generation while writing modern, safe Novus code.

2.4.1 Compiler Commands

Novus provides several commands:

`novus compile` Compile a single source file to an executable

`novus build` Build a project defined by `project.toml`

`novus test` Discover and compile tests

`novus new` Create a new project from a template

`novus fmt` Format source code

`novus clean` Remove cached build artifacts

2.4.2 Compile Options

Common options for `novus compile`:

`-o, -output` Output file name (default: `a.out`)

`-cpu` Target CPU: `68020`, `68030`, `68040`, `68060`, or `auto` (default: `auto`)

`-fpu` FPU mode: `none`, `68881`, `68882`, `68040`, `68060`, or `auto` (default: `auto`)

`-release` Enable optimizations and disable debug checks

`-O` Optimization level (0–2)

`-emit-asm` Emit assembly output only; do not assemble or link

`-emit-ir` Emit IR to stdout for debugging

`-v, -verbose` Show detailed compilation progress

The `auto` CPU setting produces a "fat binary" that detects the CPU at runtime and uses optimized code paths accordingly.

2.4.3 Debug vs Release Builds

By default, Novus compiles in debug mode:

- Array bounds checking is enabled
- Debug symbols are included
- Optimizations are minimal

For production builds, use `-release`:

```
novus compile hello.novus -o hello --release
```

Release builds are smaller and faster, but runtime errors provide less diagnostic information.

Safety Levels

For finer control, use `-safety-level`:

Level 0 Unsafe — no runtime checks (use with `-unsafe` flag)

Level 1 Basic — minimal checks

Level 2 Full — standard safety checks (default for debug)

Level 3 Paranoid — maximum checking

2.5 Project Structure

For anything beyond a single file, create a project with `novus new`:

```
novus new myproject --template cli
cd myproject
```

This creates a standard project structure:

```
myproject/
    project.toml        # Project configuration
    src/
        main.novus      # Entry point
```

2.5.1 The project.toml File

The `project.toml` file defines project metadata and build settings:

```
[package]
name = "myproject"
version = "0.1.0"

[build]
target_cpu = "68020"
fpu = "none"
```

Available build settings include:

`target_cpu` Target CPU (68020, 68030, 68040, 68060, or auto)

`fpu` FPU mode

`chipset` Target chipset (ocs, ecs, aga, or auto)

`optimization_level` Optimization level (0–2)

`output` Output file name

Build the project with:

```
novus build
```

2.6 The Standard Library

Novus includes a standard library providing common functionality:

std::core Fundamental types: `Option`, `Result`, and core traits

std::collections Data structures: `Vec`, `HashMap`, `HashSet`, `VecDeque`, `RingBuffer`, `SlotMap`, and more

std::strings String handling: `String`, `Str`, `StringBuilder`

std::memory Memory management: `MemoryBlock`, chip memory pools, slices

std::math Fixed-point arithmetic, trigonometry, vectors, easing functions

std::io Input/output: `println`, file operations

std::ffi AmigaOS bindings: Exec, DOS, Graphics, Intuition, and more

Import modules with `from`:

```
from std::collections::vec import Vec
from std::io::file import println

fn main() -> i32 {
    var names: Vec<&str> = Vec::new()
    names.push("Amiga")
    names.push("Novus")

    println("Count: {}", names.len())
    return 0
}
```

2.7 Running Tests

Novus has built-in test support. Mark test functions with the `@test` attribute:

```
from std::test::test import expect_eq

@test
fn test_addition() {
    expect_eq(2 + 2, 4, "basic addition")
}

@test
fn test_multiplication() {
    expect_eq(3 * 7, 21, "basic multiplication")
}
```

The `expect_eq` function uses overloading to work with any comparable type—no need for type-specific variants like `expect_eq_i32`.

Compile the test runner with:

```
novus test mytest.novus
```

This discovers all `@test` functions and compiles a test executable. The output tells you where the executable was created:

```
Test runner built successfully!
Output: /path/to/tests
Tests: 2 active, 0 skipped

Copy to Amiga and run to execute tests.
```

Important: The test command builds the test runner but does not execute it. You must transfer the executable to your Amiga and run it there to see test results.

2.7.1 Test Options

The `novus test` command supports several options:

`-filter` Run only tests matching a pattern

`-list` List discovered tests without building

`-b, -benchmark` Include timing information in output

`-output-dir` Specify where to write the test executable

2.8 Next Steps

You now have a working Novus installation and understand the basic workflow. The following chapters cover:

- **Chapter 3: Language Basics** — Variables, types, and expressions
- **Chapter 4: Types** — The type system in depth
- **Chapter 5: Control Flow** — Conditionals, loops, and pattern matching

When you're ready, turn the page and start learning the language itself.

Chapter 3

Language Basics

> *"Simplicity is prerequisite for reliability."*
> —Edsger W. Dijkstra

This chapter covers the fundamental building blocks of Novus: how to declare variables, what types are available, how literals work, and how expressions combine values. By the end, you'll understand the core syntax and be ready to write meaningful programs.

3.1 Comments

Novus supports two comment styles:

3.1.1 Single-Line Comments

Use `//` for comments that extend to the end of the line:

```
// This is a single-line comment
let x = 42  // Comments can appear after code
```

3.1.2 Multi-Line Comments

Use `/* */` for comments spanning multiple lines:

```
/*
 * This is a multi-line comment.
 * Useful for longer explanations.
 */
let result = compute()
```

Multi-line comments do not nest. The first `*/` closes the comment, regardless of how many `/*` appeared inside.

3.2 Variables and Mutability

Novus provides three keywords for declaring bindings:

3.2.1 Mutable Variables with `var`

Variables declared with `var` can be reassigned:

```
var x = 10
x = 20  // OK - can reassign
x = x + 5  // OK
```

Use `var` for counters, accumulators, temporary values, and any data that changes during execution.

3.2.2 Immutable Bindings with `let`

Bindings declared with `let` cannot be reassigned:

```
let x = 10
x = 20  // ERROR - cannot reassign
```

Immutable bindings make code intent clearer and allow the compiler to optimize more aggressively. Use `let` by default; switch to `var` when you need mutability.

Note that immutability applies to the binding, not necessarily to the data it refers to. For example, a `let` binding to a mutable reference (`&var T`) still allows modifying the referenced data.

3.2.3 Compile-Time Constants with `const`

Constants declared with `const` are evaluated at compile time:

```
pub const SCREEN_WIDTH: u32 = 320
pub const SCREEN_HEIGHT: u32 = 200
pub const SCREEN_SIZE: u32 = SCREEN_WIDTH * SCREEN_HEIGHT
```

Constants must be literals or constant expressions—values that the compiler can evaluate without executing code. They cannot call functions or use runtime values.

Constants are typically declared at module scope with `pub` visibility.

3.2.4 Comparison

Keyword	Reassignable	Evaluation	Typical Scope
`var`	Yes	Runtime	Function/Block
`let`	No	Runtime	Function/Block
`const`	No	Compile-time	Module

3.2.5 Type Annotations

You can explicitly annotate types:

```
let x: i32 = 42
var count: u32 = 0
```

When the type is obvious from context, annotations are optional:

```
let x = 42        // Inferred as i32
let y = 42u32     // Type suffix makes it u32
```

The compiler performs type inference within function bodies. Module-level constants typically require explicit types.

3.3 Primitive Types

Novus provides several categories of primitive types.

3.3.1 Integer Types

Novus supports signed and unsigned integers in multiple sizes:

`i8` Signed 8-bit integer (−128 to 127)

`i16` Signed 16-bit integer (−32,768 to 32,767)

`i32` Signed 32-bit integer (−2,147,483,648 to 2,147,483,647)

`i64` Signed 64-bit integer (wide range, requires two 32-bit registers on 68k)

`u8` Unsigned 8-bit integer (0 to 255)

`u16` Unsigned 16-bit integer (0 to 65,535)

`u32` Unsigned 32-bit integer (0 to 4,294,967,295)

`u64` Unsigned 64-bit integer (0 to 18,446,744,073,709,551,615)

The default integer type is `i32`. When you write a bare integer literal like `42`, it defaults to `i32` unless context requires otherwise.

3.3.2 Floating-Point Types

`f32` 32-bit IEEE 754 single-precision float

`f64` 64-bit IEEE 754 double-precision float

On 68k systems without an FPU, floating-point operations use software emulation—slow but accurate. For performance-critical math on non-FPU systems, prefer fixed-point types.

3.3.3 Fixed-Point Types

Novus provides fixed-point arithmetic for efficient fractional math on integer hardware:

`fixed16` 16-bit fixed-point (8.8 format: 8 integer bits, 8 fractional bits)

`fixed32` 32-bit fixed-point (16.16 format: 16 integer bits, 16 fractional bits)

Fixed-point types use native 68k integer instructions, making them dramatically faster than soft-float on systems without an FPU.

3.3.4 Boolean Type

The `bool` type represents truth values:

```
let flag: bool = true
let done: bool = false
```

Booleans result from comparison and logical operations. Unlike C, integers and pointers do not implicitly convert to booleans—you must use explicit comparisons.

3.4 Literals

3.4.1 Integer Literals

Integer literals support multiple bases and optional type suffixes.

Decimal

```
let x = 42
let y = 1_000_000  // Underscores improve readability
```

Underscores are ignored; they exist solely to make large numbers easier to read.

Hexadecimal

Novus supports both C-style and Amiga-style hexadecimal:

```
let color1 = 0xFF00AA  // C-style
let color2 = $FF00AA   // Amiga-style (same value)
```

Both forms are equivalent. Use whichever feels natural—Amiga developers often prefer `$`, while those from C backgrounds prefer `0x`.

Binary

Binary literals use the `%` prefix:

```
let mask = %1010     // Binary literal (value: 10)
let flags = %11110000
```

Binary literals are useful for bit patterns and masks when working with hardware registers.

Type Suffixes

Append a type suffix to specify the literal's type explicitly:

```
let a = 42u32   // u32
let b = 42i8    // i8
let c = 100u16  // u16
```

Type suffixes work with all bases:

```
let hex_byte = $FFu8
let binary_word = %1010u16
let decimal_long = 1000u32
```

Without a suffix, integer literals default to `i32`.

3.4.2 Floating-Point Literals

Floating-point literals require a decimal point:

```
let pi = 3.14
let e = 2.71828f32  // f32 with explicit suffix
```

Without a suffix, floating-point literals default to `f64`.

3.4.3 String Literals

String literals use double quotes and support escape sequences:

```
let greeting = "Hello, Amiga!"
let multiline = "Line 1\nLine 2\nLine 3"
let path = "SYS:Utilities\\"  // Escaped backslash
```

Supported escape sequences include:

`\n` Newline (line feed)

`\r` Carriage return

`\t` Horizontal tab

`\0` Null byte

`\\` Backslash

`\"` Double quote

`\'` Single quote

`\xNN` Hexadecimal byte (e.g., `\x1B` for ESC)

3.4.4 F-Strings (Formatted Strings)

F-strings embed expressions directly in string literals:

```
let x = 42
let message = f"The answer is {x}"
// Results in: "The answer is 42"
```

Expressions inside `{...}` are evaluated and converted to strings. F-strings compile to efficient string concatenation or formatting calls.

3.4.5 Character Literals

Character literals use single quotes:

```
let ch: u8 = 'A'
let newline: u8 = '\n'
let escape: u8 = '\x1B'  // ESC character
```

Character literals produce values of type `u8`. Novus does not have a separate `char` type—characters are simply 8-bit unsigned integers.

3.5 Operators

3.5.1 Arithmetic Operators

Standard arithmetic works on integer and floating-point types:

Operator	Meaning
`+`	Addition
`-`	Subtraction
`*`	Multiplication
`/`	Division
`%`	Modulus (remainder)

Division by zero is a runtime error in debug builds and undefined behavior in release builds. Always validate denominators when accepting untrusted input.

```
let sum = 10 + 20       // 30
let product = 5 * 6     // 30
let quotient = 20 / 5   // 4
let remainder = 17 % 5 // 2
```

3.5.2 Comparison Operators

Comparisons produce `bool` values:

Operator	Meaning
`==`	Equal to
`!=`	Not equal to
`<`	Less than
`>`	Greater than
`<=`	Less than or equal to
`>=`	Greater than or equal to

```
let a = 10
let b = 20
let equal = a == b      // false
let less = a < b        // true
let greater_eq = a >= b // false
```

3.5.3 Logical Operators

Logical operators combine boolean values:

Operator	Meaning
`&&`	Logical AND (short-circuiting)
`||`	Logical OR (short-circuiting)
`!`	Logical NOT

```
let x = true
let y = false
let and_result = x && y  // false
let or_result = x || y   // true
let not_result = !x      // false
```

Both `&&` and `||` short-circuit: the right-hand side is not evaluated if the result is determined by the left-hand side alone.

3.5.4 Bitwise Operators

Bitwise operators manipulate individual bits:

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT (one's complement)
«	Left shift
»	Right shift (arithmetic for signed, logical for unsigned)

```
let a = %1111  // 15
let b = %1010  // 10

let and_bits = a & b   // %1010 (10)
let or_bits = a | b    // %1111 (15)
let xor_bits = a ^ b   // %0101 (5)
let not_bits = ~a      // %0000 (-16 in two's complement)

let shifted_left = 1 << 3   // 8
let shifted_right = 16 >> 2 // 4
```

Bitwise operations compile to single 68k instructions and execute in one or two cycles—essential for custom chip programming.

3.5.5 Compound Assignment Operators

Compound assignment combines an operation with assignment:

```
var x = 10
x += 5    // x = x + 5
x -= 3    // x = x - 3
x *= 2    // x = x * 2
x /= 4    // x = x / 4
x %= 3    // x = x % 3

x &= $FF    // x = x & $FF
x |= $80    // x = x | $80
x ^= $AA    // x = x ^ $AA
x <<= 2     // x = x << 2
x >>= 1     // x = x >> 1
```

All arithmetic and bitwise operators have corresponding compound forms.

3.5.6 Increment and Decrement Operators

Novus supports both prefix and postfix increment/decrement:

```
var x = 5

++x  // Pre-increment: x becomes 6, evaluates to 6
--x  // Pre-decrement: x becomes 5, evaluates to 5

let a = x++  // Post-increment: a = 5, then x becomes 6
let b = x--  // Post-decrement: b = 6, then x becomes 5
```

Prefix operators modify the variable and return the new value. Postfix operators return the original value and then modify the variable.

3.5.7 Range Operators

Range operators create range values for iteration:

`start..end` Exclusive range (`start` to `end - 1`)

`start..=end` Inclusive range (`start` to `end`)

```
for i in 0..10 {
    // i goes from 0 to 9
}

for j in 0..=10 {
    // j goes from 0 to 10 (inclusive)
}
```

Ranges are covered in detail in Chapter 5: Control Flow.

3.6 Type Conversions

3.6.1 C-Style Casts

Use C-style cast syntax to convert between numeric types:

```
let x: u32 = 42
let y: i32 = (i32)x
let z: *u8 = (*u8)y
```

Casts can chain:

```
let w: i64 = (i64)(i32)(u32)x
```

Casts are explicit and potentially unsafe—truncation, sign changes, and pointer conversions are your responsibility. The compiler trusts that you know what you're doing.

3.6.2 The `as` Keyword

The `as` keyword provides an alternative syntax for type assertions:

```
let x = 42 as u32
```

Both cast styles are valid; choose whichever feels more natural.

3.7 References and Pointers

Novus distinguishes between references (non-null pointers checked at compile time) and raw pointers (nullable, unchecked).

3.7.1 Immutable References

Create a reference with the `&` operator:

```
var x = 10
let ref_x: &i32 = &x
```

The type is `&i32`—a reference to an `i32`. References are guaranteed non-null and automatically dereference in most contexts.

3.7.2 Mutable References

For references that allow modification, use `&var`:

```
var x = 10
let ref_x: &var i32 = &var x
*ref_x = 20  // OK - modifies x
```

The type is `&var i32`. The `&var` syntax appears both in the type annotation and when creating the reference.

Note that Novus uses `var` for mutable, not `mut` as in some other languages.

3.7.3 Raw Pointers

Raw pointers use the `*T` syntax:

```
let ptr: *u8 = (*u8)0xDFF000  // Custom chip register
```

Pointers can be null and require explicit null checks:

```
if ptr == null {
    return  // Handle null case
}
```

Raw pointers are essential for FFI and hardware access, but prefer references when possible.

3.7.4 Dereferencing

Use `*` to dereference a pointer or reference:

```
let x = 10
let ref_x = &x
let value = *ref_x  // 10
```

For pointers, dereferencing an invalid or null pointer is undefined behavior.

3.8 Arrays

3.8.1 Fixed-Size Arrays

Arrays have a fixed size known at compile time:

```
let numbers: [i32; 3] = [10, 20, 30]
```

The type `[i32; 3]` means "array of 3 `i32` values." The size is part of the type.

3.8.2 Array Literals

Create arrays with bracket syntax:

```
let primes = [2, 3, 5, 7, 11]
```

The compiler infers the size from the number of elements.

3.8.3 Repeat Syntax

Initialize arrays with a repeated value:

```
let zeros = [0; 100]  // 100 zeros
let buffer: [u8; 1024] = [0; 1024]
```

The syntax `[value; count]` creates an array of `count` elements, each initialized to `value`.

3.8.4 Indexing

Access array elements with bracket indexing:

```
let numbers = [10, 20, 30]
let first = numbers[0u32]   // 10
let second = numbers[1u32]  // 20
```

Indices must be unsigned integers. In debug builds, out-of-bounds access triggers a panic. In release builds, bounds checking may be elided for performance.

3.9 Tuples

3.9.1 Tuple Types

Tuples group multiple values of potentially different types:

```
let point: (i32, i32) = (100, 200)
let rgb: (u8, u8, u8) = (255, 128, 64)
```

The type `(i32, i32)` means "a tuple of two `i32` values."

3.9.2 Tuple Literals

Create tuples with parenthesized, comma-separated values:

```
let color = (192, 96, 32)
```

3.9.3 Destructuring

Extract tuple elements with destructuring:

```
let (r, g, b) = get_rgb()
```

This binds `r`, `g`, and `b` to the tuple's three elements. Destructuring works in `let` and `var` bindings.

3.9.4 Returning Multiple Values

Tuples enable returning multiple values from functions:

```
fn get_rgb() -> (i32, i32, i32) {
    return (255, 128, 64)
```

```
}
```

3.10 Expressions and Statements

In Novus, most constructs are expressions—they produce values.

3.10.1 Expression vs Statement

An expression evaluates to a value:

```
let x = 2 + 2  // Expression: 4
```

A statement performs an action without producing a value:

```
x = 10  // Assignment statement
```

However, even assignments are expressions in Novus, allowing chained assignment:

```
var a = 0
var b = 0
a = b = 5  // Both a and b become 5
```

3.10.2 Block Expressions

Blocks (sequences of statements enclosed in braces) can be expressions if they end with an expression:

```
let x = {
    let a = 10
    let b = 20
    a + b  // No semicolon - this is the block's value
}
// x = 30
```

If a block ends with a semicolon, it produces no value (unit type).

3.11 Summary

This chapter covered Novus's fundamental syntax:

- **Variables:** `var` for mutable, `let` for immutable, `const` for compile-time constants
- **Types:** Integers (`i8`–`i64`, `u8`–`u64`), floats (`f32`, `f64`), fixed-point (`fixed16`, `fixed32`), and `bool`
- **Literals:** Decimal, hex (`0x`/`$`), binary (`%`), strings, f-strings, and characters
- **Operators:** Arithmetic, comparison, logical, bitwise, and compound assignment
- **References:** `&T` (immutable), `&var T` (mutable), `*T` (raw pointers)
- **Arrays:** Fixed-size with type `[T; N]`, indexed with unsigned integers
- **Tuples:** Group values with `(T1, T2, ...)`, destructure with `let (a, b) = ...`

With these building blocks, you can write meaningful programs. The next chapter explores the type system in depth, covering structs, enums, and pattern matching.

Chapter 4

Types

"A type system is a syntactic method for enforcing levels of abstraction in programs."
— John C. Reynolds

Novus features a static, strong type system designed to provide safety without sacrificing the control needed for systems programming on the Amiga. Every value has a known type at compile time, enabling the compiler to catch errors early and generate efficient 68k machine code.

This chapter explores Novus's type system in depth: primitive types, composite types (structs and enums), traits for polymorphism, pattern matching for working with data, and generics for code reuse.

4.1 Type System Overview

Novus's type system is built on these principles:

- **Static typing** — all types known at compile time
- **Strong typing** — no implicit conversions between unrelated types
- **Explicit conversions** — casts must be written explicitly
- **Zero-cost abstractions** — high-level types compile to efficient machine code
- **Null safety** — no null pointers; use `Option<T>` instead
- **Error handling** — errors are values via `Result<T, E>`

Every expression in Novus has a type, and the compiler verifies that types are used correctly throughout your program.

4.2 Primitive Types

Novus provides a set of built-in primitive types (covered in Chapter 2). Here's a quick reference:

4.3 Structs

Structs are composite types that group related data together. They are the primary way to create custom types in Novus.

Type	Description
`i8`, `i16`, `i32`, `i64`	Signed integers
`u8`, `u16`, `u32`, `u64`	Unsigned integers
`bool`	Boolean (`true` or `false`)
`f32`, `f64`	Floating-point (soft-float on 68k without FPU)
`fixed16`, `fixed32`	Fixed-point (8.8 and 16.16 format)
`*T`	Raw pointer to type `T` (can be null)
`&T`, `&var T`	References (guaranteed non-null)

Table 4.1: Novus Primitive Types

4.3.1 Declaring Structs

A struct declaration defines a new type with named fields:

```
struct Point {
    x: i32,
    y: i32,
}

struct Color {
    r: u8,
    g: u8,
    b: u8,
    a: u8,
}
```

Each field has a name and a type. Fields are separated by commas, and a trailing comma is allowed.

4.3.2 Struct Visibility

By default, structs are private to the module where they're defined. Use `pub` to make them public:

```
pub struct Point {
    x: i32,
    y: i32,
}

internal struct Config {
    setting: bool,
}

struct Private {
    data: i32,
}
```

Important: Regardless of struct visibility, *all fields are always private*. Novus does not support field-level visibility modifiers. To expose field values, provide accessor methods in an `impl` block.

4.3.3 Creating Struct Instances

Create instances using struct literal syntax:

```
let origin = Point { x: 0, y: 0 };
let pixel = Point { x: 100, y: 200 };

let red = Color { r: 255, g: 0, b: 0, a: 255 };
```

All fields must be initialized. The order of fields in the literal doesn't matter.

4.3.4 Accessing Fields

Access struct fields using dot notation:

```
let p = Point { x: 10, y: 20 };
let x_coord = p.x;  // 10
let y_coord = p.y;  // 20
```

To modify fields, the binding must be mutable:

```
var p = Point { x: 10, y: 20 };
p.x = 15;
p.y = 25;
```

4.3.5 Struct Update Syntax

The spread operator `..` allows creating a new struct from an existing one, updating specific fields:

```
let p1 = Point { x: 10, y: 20 };
let p2 = Point { x: 100, ..p1 };  // { x: 100, y: 20 }
let p3 = Point { y: 200, ..p1 };  // { x: 10, y: 200 }
let p4 = Point { ..p1 };          // { x: 10, y: 20 }
```

The `..expr` must appear last in the literal and copies all unspecified fields from the given expression.

4.3.6 Generic Structs

Structs can be parameterized with type parameters:

```
struct Pair<T, U> {
    first: T,
    second: U,
}

let int_pair = Pair { first: 42, second: 100 };
let mixed = Pair { first: 10, second: "hello" };
```

The compiler infers type arguments from usage, or you can specify them explicitly:

```
let explicit: Pair<i32, i32> = Pair { first: 1, second: 2 };
```

4.3.7 Const Generic Parameters

Structs can also have compile-time constant parameters:

```
struct Buffer<const N: u32> {
    data: [u8; N],
    len: u32,
}

let small_buf: Buffer<64> = Buffer {
    data: [0; 64],
    len: 0
};
let large_buf: Buffer<1024> = Buffer {
    data: [0; 1024],
    len: 0
};
```

Const parameters must be integer types and are known at compile time.

4.3.8 Where Clauses

Struct definitions can have `where` clauses to constrain generic parameters:

```
struct Container<T> where T: Clone {
    value: T,
    backup: T,
}
```

This requires that any type `T` used with `Container` must implement the `Clone` trait.

4.4 Implementation Blocks

Methods and associated functions are defined in `impl` blocks:

```
impl Point {
    fn new(x: i32, y: i32) -> Point {
        Point { x: x, y: y }
    }

    fn distance(&self) -> f32 {
        let dx = self.x as f32;
        let dy = self.y as f32;
        sqrt(dx * dx + dy * dy)
    }

    fn translate(&var self, dx: i32, dy: i32) {
        self.x = self.x + dx;
        self.y = self.y + dy;
    }
}
```

4.4.1 Self Parameter Variants

Methods can take `self` in different forms:

```
impl Point {
    // Immutable borrow - reads only
    fn magnitude(&self) -> i32 {
```

Form	Meaning
`&self`	Immutable borrow; method reads but doesn't modify
`&var self`	Mutable borrow; method can modify the instance
`self`	Takes ownership; instance consumed by call
`consuming self`	Explicit ownership transfer; semantically identical to `self`

Table 4.2: Self Parameter Forms

```
        self.x * self.x + self.y * self.y
    }

    // Mutable borrow - modifies in place
    fn scale(&var self, factor: i32) {
        self.x = self.x * factor;
        self.y = self.y * factor;
    }

    // Consumes self - takes ownership
    fn into_tuple(self) -> (i32, i32) {
        (self.x, self.y)
    }

    // Explicit consuming - same as self
    fn consume(consuming self) {
        // self is consumed
    }
}
```

4.4.2 Associated Functions

Functions without a `self` parameter are associated functions (like static methods):

```
impl Point {
    fn origin() -> Point {
        Point { x: 0, y: 0 }
    }

    fn from_polar(r: f32, theta: f32) -> Point {
        Point {
            x: (r * cos(theta)) as i32,
            y: (r * sin(theta)) as i32,
        }
    }
}

let origin = Point::origin();
let p = Point::from_polar(10.0, 3.14159 / 4.0);
```

4.4.3 Generic Implementations

Implementations can be generic:

```
impl<T, U> Pair<T, U> {
    fn new(first: T, second: U) -> Pair<T, U> {
        Pair { first: first, second: second }
    }

    fn get_first(&self) -> &T {
        &self.first
    }
}

impl<T> Pair<T, T> where T: Clone {
    fn duplicate_first(self) -> Pair<T, T> {
        let copy = self.first.clone();
        Pair { first: self.first, second: copy }
    }
}
```

4.4.4 Multiple Impl Blocks

A type can have multiple `impl` blocks:

```
impl Point {
    fn new(x: i32, y: i32) -> Point {
        Point { x: x, y: y }
    }
}

impl Point {
    fn distance(&self) -> f32 {
        sqrt((self.x * self.x + self.y * self.y) as f32)
    }
}
```

This is useful for organizing code or when implementing different traits.

4.5 Enums

Enumerations define a type by enumerating its possible variants. Each variant can carry associated data.

4.5.1 Unit Variants

Simple enums list named constants:

```
enum Direction {
    North,
    South,
    East,
    West,
}

let heading = Direction::North;
```

4.5.2 Tuple Variants

Variants can carry data as tuples:

```
enum Command {
    Quit,
    Move(i32, i32),
    Write(i32),
    ChangeColor(u8, u8, u8),
}

let cmd1 = Command::Quit;
let cmd2 = Command::Move(10, 20);
let cmd3 = Command::ChangeColor(255, 0, 0);
```

Each variant can have a different number and type of fields.

4.5.3 Generic Enums

Enums can be parameterized with type parameters:

```
enum Option<T> {
    Some(T),
    None,
}

enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

These are the foundation of Novus's error handling and null safety (covered in detail later).

4.5.4 Enum Limitations

Important: Novus enums support only *unit variants* and *tuple variants*. Struct-style variants are **not supported**:

```
// NOT SUPPORTED - struct variants
enum Message {
    Move { x: i32, y: i32 },  // ERROR!
}

// Instead, use tuple variants:
enum Message {
    Move(i32, i32),  // OK
}
```

If you need named fields, define a separate struct:

```
struct MoveData {
    x: i32,
    y: i32,
}

enum Message {
    Move(MoveData),
```

```
    Quit,
}
```

4.6 Traits

Traits define shared behavior across types. They're similar to interfaces in other languages but more powerful.

4.6.1 Defining Traits

A trait declares a set of methods that implementing types must provide:

```
trait Clone {
    fn clone(&self) -> Self
}

trait Eq {
    fn eq(&self, other: &Self) -> bool
}

trait Hash {
    fn hash(&self) -> u32
}
```

4.6.2 Default Implementations

Traits can provide default implementations:

```
trait Printable {
    fn print(&self) {
        // Default implementation
        println("Printable object");
    }

    fn debug_print(&self);  // Must be implemented
}
```

Types implementing the trait can use the default or override it.

4.6.3 Implementing Traits

Use `impl TraitName for TypeName` to implement a trait:

```
impl Clone for Point {
    fn clone(&self) -> Point {
        return Point { x: self.x, y: self.y }
    }
}

impl Eq for Point {
    fn eq(&self, other: &Point) -> bool {
        return self.x == other.x && self.y == other.y
    }
}
```

Once implemented, you can call trait methods:

```
let p1 = Point { x: 10, y: 20 };
let p2 = p1.clone();
let s = p1.to_string();
```

4.6.4 Built-in Traits

Novus provides several important built-in traits:

Trait	Purpose
`Drop`	Custom cleanup when value goes out of scope
`Clone`	Explicit duplication of values
`Copy`	Marker trait for types that can be copied implicitly
`Eq`	Equality comparison
`Hash`	Hashing for collections
`From<T>`	Type conversion from `T`
`Default`	Provides a default value

Table 4.3: Core Built-in Traits

The Drop Trait

`Drop` provides custom cleanup logic:

```
struct FileHandle {
    fd: i32,
}

impl Drop for FileHandle {
    fn drop(&var self) {
        close_file(self.fd);
    }
}
```

When a `FileHandle` goes out of scope, `drop` is called automatically.

The From Trait

`From<T>` enables type conversions:

```
impl From<i32> for Point {
    fn from(value: i32) -> Point {
        Point { x: value, y: value }
    }
}

let p = Point::from(42);  // Point { x: 42, y: 42 }
```

4.6.5 Trait Bounds

Traits can be used as bounds on generic parameters:

```
fn are_equal<T>(a: &T, b: &T) -> bool where T: Eq {
    return a.eq(b)
}

fn clone_if_equal<T>(a: &T, b: &T) -> Option<T> where T: Clone +
   Eq {
    if a.eq(b) {
        return Option::Some(a.clone())
    }
    return Option::None
}
```

The first function requires `T` to implement `Eq`. The second requires both `Clone` and `Eq`, using `+` to combine bounds.

4.7 Pattern Matching

Pattern matching is a powerful feature for destructuring and inspecting data. It's the primary way to work with enums and extract data from composite types.

4.7.1 Pattern Types

Novus supports several pattern forms:

Literal Patterns

Match exact values:

```
42          // Matches the number 42
true        // Matches boolean true
"hello"     // Matches the string "hello"
null        // Matches null pointer
```

Identifier Patterns

Bind a value to a name:

```
x           // Binds value to variable x
count       // Binds value to variable count
```

Wildcard Pattern

Ignore a value:

```
_           // Matches anything, discards value
```

Variant Patterns

Destructure enum variants:

```
Option::Some(x)           // Matches Some, binds inner value to x
Option::None              // Matches None
```

```
Command::Move(x, y)        // Matches Move, binds coordinates
Result::Err(_)             // Matches Err, discards error value
```

Or Patterns

Match multiple patterns:

```
1 | 2 | 3                  // Matches 1, 2, or 3
Direction::North | Direction::South  // Matches either variant
```

Reference Patterns

Match through references:

```
&x          // Matches a reference, binds referent to x
&42         // Matches reference to 42
```

Guarded Patterns

Add conditional guards:

```
x if x > 0               // Matches any value > 0
Some(x) if x < 100       // Matches Some with value < 100
```

4.7.2 Match Expressions

The `match` expression compares a value against patterns:

```
match value {
    Pattern1 => expression1,
    Pattern2 => expression2,
    Pattern3 if guard => expression3,
    _ => default_expression,
}
```

Each arm consists of a pattern, optional guard, and an expression. The first matching pattern's expression is evaluated.

Basic Matching

```
let number = 42;
let description = match number {
    0 => "zero",
    1 => "one",
    2 | 3 | 5 | 7 => "small prime",
    42 => "the answer",
    _ => "something else",
};
```

Matching Enums

```
let cmd = Command::Move(10, 20);

match cmd {
    Command::Quit => {
        println("Exiting");
        exit();
    },
    Command::Move(x, y) => {
        println("Moving to ({}, {})", x, y);
        move_cursor(x, y);
    },
    Command::Write(ch) => {
        println("Writing character {}", ch);
        write_char(ch);
    },
    Command::ChangeColor(r, g, b) => {
        set_color(r, g, b);
    },
}
```

Pattern Guards

Guards add conditional logic to patterns:

```
let number = 15;

match number {
    x if x < 0 => println("negative"),
    x if x == 0 => println("zero"),
    x if x < 10 => println("small positive"),
    x if x < 100 => println("medium positive"),
    _ => println("large positive"),
}
```

Exhaustiveness

Match expressions must be *exhaustive* — they must cover all possible values:

```
// ERROR: non-exhaustive
match direction {
    Direction::North => "north",
    Direction::South => "south",
    // Missing East and West!
}

// OK: exhaustive with wildcard
match direction {
    Direction::North => "north",
    Direction::South => "south",
    _ => "other",
}

// OK: exhaustive with all variants
```

```
match direction {
    Direction::North => "north",
    Direction::South => "south",
    Direction::East => "east",
    Direction::West => "west",
}
```

The compiler verifies exhaustiveness and reports errors if any case is unhandled.

4.7.3 If Let Expressions

For simple matches with one pattern, `if let` is more concise:

```
let opt = Option::Some(42);

if let Option::Some(x) = opt {
    println("Got value: {}", x);
}
```

This is equivalent to:

```
match opt {
    Option::Some(x) => {
        println("Got value: {}", x);
    },
    _ => {},
}
```

You can add an `else` clause:

```
if let Option::Some(x) = opt {
    println("Got value: {}", x);
} else {
    println("No value");
}
```

4.7.4 Let Else Expressions

`let else` combines pattern matching with early return for error handling:

```
fn process(opt: Option<i32>) -> i32 {
    let Option::Some(x) = opt else {
        return 0;  // Must diverge (return/break/continue)
    };

    x * 2
}
```

The `else` block must diverge (never return normally) via `return`, `break`, `continue`, or calling a function that never returns.

This is particularly useful for unwrapping `Option` and `Result` values:

```
fn read_config(path: &str) -> Result<Config, Error> {
    let Result::Ok(file) = open_file(path) else {
        return Result::Err(Error::FileNotFound);
```

```
    };

    let Result::Ok(contents) = read_file(&file) else {
        return Result::Err(Error::ReadFailed);
    };

    parse_config(&contents)
}
```

4.8 Option and Result

Novus eliminates null pointer errors and exception-based error handling by using two core types: `Option<T>` and `Result<T, E>`.

4.8.1 Option<T>

`Option<T>` represents an optional value — either `Some(T)` or `None`:

```
enum Option<T> {
    Some(T),
    None,
}
```

Use `Option` when a value might be absent:

```
fn find_user(id: u32) -> Option<User> {
    if user_exists(id) {
        Option::Some(get_user(id))
    } else {
        Option::None
    }
}
```

Working with Option

Check if a value exists:

```
let opt = Option::Some(42);

if opt.is_some() {
    println("Has value");
}

if opt.is_none() {
    println("No value");
}
```

Extract the value:

```
// Pattern matching (safe)
match opt {
    Option::Some(x) => println("Value: {}", x),
    Option::None => println("No value"),
}
```

```
// if let (safe)
if let Option::Some(x) = opt {
    println("Value: {}", x);
}

// unwrap (unsafe - panics on None)
let x = opt.unwrap();

// unwrap_or (safe - provides default)
let x = opt.unwrap_or(0);
```

Option Methods

Common `Option` methods:

Method	Description
`is_some() -> bool`	Returns `true` if `Some`
`is_none() -> bool`	Returns `true` if `None`
`unwrap() -> T`	Returns value, panics on `None`
`unwrap_or(T) -> T`	Returns value or default
`ok_or(E) -> Result<T,E>`	Converts to `Result`

Table 4.4: Common Option Methods

4.8.2 Result<T, E>

`Result<T, E>` represents either success (`Ok(T)`) or failure (`Err(E)`):

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

Use `Result` for operations that can fail:

```
fn parse_number(s: &str) -> Result<i32, ParseError> {
    if is_valid_number(s) {
        Result::Ok(parse(s))
    } else {
        Result::Err(ParseError::InvalidFormat)
    }
}
```

Working with Result

Check for success or failure:

```
let result = parse_number("42");

if result.is_ok() {
    println("Parse succeeded");
}
```

```
if result.is_err() {
    println("Parse failed");
}
```

Extract the value or error:

```
// Pattern matching (safe)
match result {
    Result::Ok(n) => println("Number: {}", n),
    Result::Err(e) => println("Error: {}", e),
}

// if let for success case
if let Result::Ok(n) = result {
    println("Number: {}", n);
}

// if let for error case
if let Result::Err(e) = result {
    println("Error: {}", e);
}

// unwrap (unsafe - panics on Err)
let n = result.unwrap();

// unwrap_or (safe - provides default)
let n = result.unwrap_or(0);
```

Result Methods

Common `Result` methods:

Method	Description
`is_ok() -> bool`	Returns `true` if `Ok`
`is_err() -> bool`	Returns `true` if `Err`
`unwrap() -> T`	Returns value, panics on `Err`
`unwrap_or(T) -> T`	Returns value or default
`ok() -> Option<T>`	Converts to `Option`, discards error
`err() -> Option<E>`	Gets error as `Option`

Table 4.5: Common Result Methods

4.8.3 The ? Operator

The `?` operator provides early return on error:

```
fn read_number_from_file(path: &str) -> Result<i32, Error> {
    let file = open_file(path)?;         // Returns Err early
    let contents = read_file(&file)?;    // Returns Err early
    let number = parse_number(&contents)?;  // Returns Err early
    Result::Ok(number)
}
```

The `?` operator:

1. Checks if the `Result` is `Err`
2. If `Err`, returns the error immediately
3. If `Ok`, unwraps the value and continues

This is equivalent to:

```
fn read_number_from_file(path: &str) -> Result<i32, Error> {
    let file = match open_file(path) {
        Result::Ok(f) => f,
        Result::Err(e) => return Result::Err(e),
    };

    let contents = match read_file(&file) {
        Result::Ok(c) => c,
        Result::Err(e) => return Result::Err(e),
    };

    let number = match parse_number(&contents) {
        Result::Ok(n) => n,
        Result::Err(e) => return Result::Err(e),
    };

    Result::Ok(number)
}
```

The `?` operator works with both `Result` and `Option`:

```
fn get_first_element(vec: &Vec<i32>) -> Option<i32> {
    let first = vec.get(0)?;  // Returns None early if empty
    Option::Some(*first * 2)
}
```

4.9 Generics

Generics enable code reuse by parameterizing types and functions with type variables.

4.9.1 Generic Functions

Functions can be generic over types:

```
fn identity<T>(value: T) -> T {
    value
}

let x = identity(42);         // T = i32
let s = identity("hello");    // T = &str
```

Multiple type parameters:

```
fn pair<T, U>(first: T, second: U) -> (T, U) {
    (first, second)
}

let p = pair(42, "hello");  // T = i32, U = &str
```

4.9.2 Generic Structs

Structs can be generic (as shown earlier):

```
struct Vec<T> {
    data: *var T,
    len: u32,
    capacity: u32,
}

impl<T> Vec<T> {
    fn new() -> Vec<T> {
        Vec { data: null, len: 0, capacity: 0 }
    }

    fn push(&var self, value: T) {
        // Implementation
    }

    fn get(&self, index: u32) -> Option<&T> {
        if index < self.len {
            Option::Some(&self.data[index])
        } else {
            Option::None
        }
    }
}
```

4.9.3 Const Generic Parameters

Functions and types can be parameterized with compile-time constants:

```
fn create_array<const N: u32>() -> [i32; N] {
    [0; N]
}

let arr1 = create_array::<10>();   // [i32; 10]
let arr2 = create_array::<100>();  // [i32; 100]

struct Matrix<const ROWS: u32, const COLS: u32> {
    data: [f32; ROWS * COLS],
}
```

Const parameters must be integer types and known at compile time.

4.9.4 Where Clauses

Generic parameters can be constrained with `where` clauses:

```
fn hash_clone<T>(value: &T) -> u32 where T: Clone + Hash {
    let copy = value.clone()
    return copy.hash()
}
```

Where clauses can appear on:

- Function definitions

- Struct definitions
- Impl blocks
- Trait definitions

Multiple constraints with +:

```
fn process<T>(value: T) where T: Clone + Eq + Hash {
    // T must implement Clone, Eq, and Hash
}
```

4.9.5 Turbofish Syntax

Type arguments can be specified explicitly using turbofish syntax ::<>:

```
let vec = Vec::<i32>::new();
let result = parse::<u32>("42");
let pair = Pair::<i32, bool>::new(10, true);
```

This is necessary when the compiler cannot infer type arguments:

```
// Ambiguous - compiler doesn't know T
let vec = Vec::new();  // ERROR

// Explicit - compiler knows T = i32
let vec = Vec::<i32>::new();  // OK
```

4.9.6 Generic Constraints in Practice

Here's a complete example showing generic constraints:

```
trait Comparable {
    fn compare(&self, other: &Self) -> i32;
}

fn find_max<T>(items: &[T]) -> Option<&T>
    where T: Comparable
{
    if items.len() == 0 {
        return Option::None;
    }

    var max = &items[0];
    var i = 1;

    while i < items.len() {
        if items[i].compare(max) > 0 {
            max = &items[i];
        }
        i = i + 1;
    }

    Option::Some(max)
}
```

```
impl Comparable for i32 {
    fn compare(&self, other: &i32) -> i32 {
        if *self < *other {
            -1
        } else if *self > *other {
            1
        } else {
            0
        }
    }
}

let numbers = [10, 5, 20, 15];
let max = find_max(&numbers);  // Some(&20)
```

4.10 Type Coercions and Casts

Novus has strict type checking with limited implicit conversions.

4.10.1 Integer Widening

Smaller integer types can be implicitly widened to larger ones in some contexts:

```
let x: i8 = 42;
let y: i32 = x;  // Implicit widening OK (i8 -> i32)
```

4.10.2 Integer Narrowing

Narrowing requires explicit casts:

```
let x: i32 = 1000;
let y: i8 = x as i8;  // Explicit cast required (truncates)
```

4.10.3 Reference to Pointer

References can be converted to raw pointers:

```
let x = 42;
let r: &i32 = &x;
let p: *i32 = r as *i32;  // Reference to pointer
```

4.10.4 Array to Pointer

Arrays decay to pointers in some contexts:

```
let arr = [1, 2, 3, 4, 5];
let ptr: *i32 = arr as *i32;  // Array to pointer
```

4.10.5 Explicit Casts

Use `as` for explicit type conversions:

```
let x: i32 = 42;
let y: f32 = x as f32;     // Int to float
let z: u32 = x as u32;     // Signed to unsigned
let b: u8 = (x & 0xFF) as u8;   // Truncation
```

Always prefer explicit casts to make your intent clear.

4.11 Type Inference

Novus uses type inference to reduce verbosity while maintaining static typing.

4.11.1 Local Variable Inference

The `let` keyword allows the compiler to infer types:

```
let x = 42;            // Inferred as i32
let s = "hello";       // Inferred as &str
let v = Vec::new();    // ERROR: cannot infer T
let v = Vec::<i32>::new();  // OK: explicit type argument
```

You can always specify types explicitly:

```
let x: i32 = 42;
let s: &str = "hello";
let v: Vec<i32> = Vec::new();
```

4.11.2 Function Return Type Inference

Function return types must be explicitly declared:

```
// ERROR: missing return type
fn add(a: i32, b: i32) {
    a + b
}

// OK: explicit return type
fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

4.11.3 Generic Type Inference

Generic type arguments are often inferred from usage:

```
let mut vec = Vec::<i32>::new();
vec.push(42);  // T inferred as i32

let opt = Option::Some(42);  // T inferred as i32
```

When inference fails, use turbofish syntax:

```
let result = parse::<i32>("42");
```

4.12 Summary

Novus's type system provides:

- **Structs** for grouping related data with private fields
- **Enums** for sum types with unit and tuple variants
- **Traits** for shared behavior and polymorphism
- **Pattern matching** for safe data inspection and destructuring
- **Option and Result** for null safety and error handling
- **Generics** for code reuse with type and const parameters
- **Type inference** to reduce verbosity
- **Explicit casts** for type conversions

These features combine to create a type system that catches errors at compile time while remaining expressive and efficient. The emphasis on explicitness (no hidden allocations, no implicit conversions) and safety (no null, errors as values) makes Novus code predictable and robust.

In the next chapter, we'll explore Novus's memory management model, including ownership, borrowing, and the interaction with AmigaOS memory pools.